

# VisionWaste — Full System Overview

Vision-Based Hygienic Risk Prediction and Animal Deterrence System

**Goal:** Avoid inconvenience from irregular waste    **Document type:** Technical system overview

**Date:** 30 August 2026

VisionWaste watches public bins with cameras and phones, classifies waste, detects animals, estimates fill level, scores outside-bin litter, combines weather, then computes a readable hygienic risk. Staff get alerts, maps, forecasts, and collection routes. Citizens find the nearest available bin.

## 1. What the system does

From one camera image (ESP32-CAM, phone, or admin upload), the system:

1. Classifies waste as **organic** or **non-organic**
2. Detects animals: **cat, crow, dog, monkey**
3. Estimates bin fill: **Empty / Half / Overflow**
4. Optionally scores **litter outside the bin** (LSI 0–100)
5. Pulls **weather** (temperature / humidity)
6. Computes a **hygienic risk** level with an explicit case and message
7. Stores the capture, raises **alerts**, can play a **speaker/alarm**, and supports **forecasting** and **collection routes**

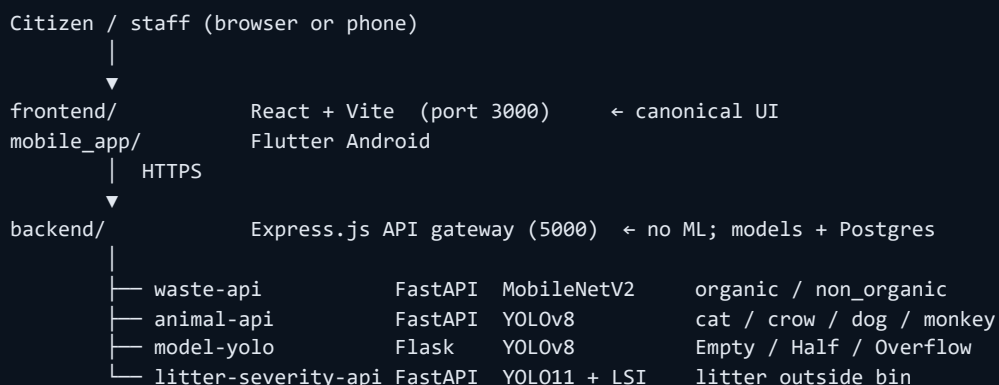
Citizens use a public map. Staff use a dark web dashboard and a Flutter Android app.

## 2. Architecture (separated services)

Machine learning never runs inside the Express gateway. Each model is its own process so PyTorch / TensorFlow stay isolated.

**Rule:** the frontend talks only to backend. The backend talks only to model microservices over HTTP.

The folder web/ is a deprecated copy of an older React app. Develop and deploy **frontend/** only.



```
VisionWaste/bridge/  laptop on same Wi-Fi as ESP32-CAM
| GET LAN snapshot → POST HTTPS /predict
▼
ESP32-CAM + speaker (buzzer / PCM5102 DAC / DFPlayer)
```

### 3. End-to-end capture flow

1. Image arrives at **POST /predict** (multipart JPEG/PNG).
2. Gateway fans out to **waste + animal** (and **bin-fill** if MODEL\_YOLO\_URL is set).
3. Weather is taken from request GPS, else the bin's coordinates, else Colombo defaults (**6.9271, 79.8612**).
4. **Risk engine** combines waste + animals + weather.
5. If Postgres is on, a **capture** row is saved (X-Capture-Id header).
6. Alerts can be auto-derived from that capture.
7. For ESP32 uploads, **auto audio** can queue DFPlayer tracks on HIGH / CRITICAL risk.
8. The laptop **bridge** can also hit the ESP32 /alarm when risk is HIGH / CRITICAL.

Source types: esp32 | mobile | admin.

### 4. AI models

| Model    | Technology                   | Classes                | Role                    |
|----------|------------------------------|------------------------|-------------------------|
| Waste    | TensorFlow MobileNetV2 (.h5) | organic, non_organic   | Rotting / hygiene input |
| Animal   | YOLOv8                       | cat, crow, dog, monkey | Deterrence + HIGH risk  |
| Bin fill | YOLOv8                       | Empty, Half, Overflow  | Collection priority     |
| Litter   | YOLO11s + LSI                | single class: litter   | Outside-bin mess score  |

#### Evaluation snapshot

From evaluation\_results/report.md:

- Waste: **97.58%** test accuracy (372 images)
- Animal: **mAP50 0.88**, mAP50-95 0.61
- Bin fill: mAP50 **0.59** (validation from checkpoint; weaker than the others)
- Risk engine: **6/6** scripted scenarios pass
- Litter weights were missing at last eval, so LSI was skipped

Rotting estimate (organic only): baseline **36 hours**, shortened by heat and humidity.

### 5. Hygienic risk engine (core logic)

This is **rule-based**, not a neural net. Cases are explicit so the dashboard can show *why*.

| Case   | Condition                | Level |
|--------|--------------------------|-------|
| CASE 3 | Organic + any animal     | HIGH  |
| Extra  | Non-organic + any animal | HIGH  |

| Case   | Condition                                         | Level  |
|--------|---------------------------------------------------|--------|
| CASE 2 | Organic + high temp AND high humidity, no animals | MEDIUM |
| CASE 1 | Organic, no animals, normal weather               | LOW    |
| Extra  | Non-organic, no animals                           | LOW    |

Thresholds (environment, defaults): **30°C** and **70%** humidity.

**CASE 4 CRITICAL** (mixed waste + animals) is **not implemented**. The waste model is binary, so “mixed” cannot be inferred from one image.

## Forecasting

Forecasting is not a separate ML model. It **replays** this same engine on OpenWeather (or stub) future slots, assuming:

- waste type stays the same until cleaned
- animal count stays at the *current* value (it does not predict new animals)
- hours-since-clean grows linearly

## 6. Litter Severity Index (LSI)

Detect litter instances, then fuse:

**LSI**  $\approx 0.5 \times \text{count} + 0.32 \times \text{area} + 0.18 \times \text{spread}$  (tunable in `lsi_config.yaml`)

- LOW  $\leq 30$
- MEDIUM  $\leq 53$
- HIGH above that

Optional `bin_polygon` drops detections whose centroid is **inside** the bin so only **outside-bin** litter counts.

UI: dashboard `/litter-severity`. API: **POST** `/litter-severity`.

## 7. Hardware and the LAN problem

Railway **cannot** reach private IPs (`10.x.x.x`). Mixed-content also blocks an `https://` frontend from fetching an `http://` ESP32.

### Laptop bridge (VisionWaste/bridge/)

- Polls `http://<ESP32>/capture`
- POSTs JPEG to Railway POST `/predict`
- Sends a stable `bridge_instance_id` (file `.bridge_id`)
- Relays Test speaker and auto-alarm
- Must send `esp32_id` matching the Admin device row

### ESP32

- Snapshot: GET `/capture`
- Speaker: `/speaker/test`, `/alarm`, `/speaker/status`

- Optional PCM5102 I2S DAC (GPIO 14 / 13 / 15)
- Optional DFPlayer via command queue (PLAY\_AUDIO / STOP\_AUDIO)

Auto audio after predict

Controlled by AUTO\_AUDIO\_ON\_PREDICT:

- HIGH → track 2
- CRITICAL → track 3
- LOW / MEDIUM → none
- Cooldown default **60 seconds** per ESP32

Bin types

- **smart** — camera / IoT captures
- **virtual** — map pin + manual fill (no camera)

8. Backend (Express gateway)

Works **with or without** Postgres. Without DATABASE\_URL, /predict still works; /captures\* and /devices\* return **503**.

| Area               | Endpoints                                                          |
|--------------------|--------------------------------------------------------------------|
| Auth               | POST /auth/register, POST /auth/login (JWT; optional admin invite) |
| Infer              | POST /predict, POST /litter-severity                               |
| Bins               | GET /devices, /map, /nearest, CRUD (admin), latest image/history   |
| Captures           | GET /captures, image blobs                                         |
| Alerts             | GET /alerts, PATCH /alerts/:id (admin workflow)                    |
| Weather / forecast | GET /weather, GET /forecast                                        |
| Collection         | POST /collection/plan — ordered Half + Overflow stops              |
| Bridge             | /bridge/speaker-pending, /bridge/speaker-ack                       |
| ESP32 commands     | /devices/commands poll + ack                                       |
| Public demo        | GET /api/waste-data (holiday-adjusted fill demo)                   |
| Health             | GET /health (gateway + models + DB)                                |

Roles: user | admin.

9. Database (Postgres + Sequelize)

| Table           | Purpose                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------|
| users           | name, email, password hash, role                                                          |
| devices         | bin name, GPS, esp32_id, bridge ID, smart/virtual, camera URL, speaker pending, last seen |
| device_commands | queued PLAY_AUDIO / STOP_AUDIO for ESP32                                                  |

| Table       | Purpose                                                 |
|-------------|---------------------------------------------------------|
| captures    | image, fill, waste, animals, weather, risk, source, GPS |
| predictions | YOLO boxes per capture                                  |
| alerts      | derived operational alerts + admin status               |

Alert statuses: open → acknowledged → actioned | rejected | dismissed.

Alert reasons (highest first): CRITICAL risk, HIGH risk, animal + MEDIUM (buzzer), overflow ( $\geq 72\%$  or Overflow label), animal info.

## 10. Web dashboard (frontend/)

### Public (no login)

- / landing — find nearest bin, map, waste guide
- /login, /register

### Authenticated (dark shell)

| Route               | Feature                                       |
|---------------------|-----------------------------------------------|
| /dashboard          | System KPIs                                   |
| /live-monitoring    | Live camera / latest snapshot                 |
| /hygienic-risk      | Risk cases, rotting, weather                  |
| /bin-level-detector | Upload / ESP32 fill detection                 |
| /map                | All bins with fill/risk                       |
| /bins, /bins/:id    | Status list + detail                          |
| /animals            | Animal detections                             |
| /litter-severity    | LSI upload / result                           |
| /forecast           | 24h risk timeline                             |
| /alerts             | Alert feed + badge                            |
| /reports            | Reports                                       |
| /history            | Capture history                               |
| /mobile-report      | Phone-style photo report (source_type=mobile) |
| /speaker            | Speaker check (via bridge)                    |
| /admin              | Settings, bins, hero/promo                    |

Maps: **MapTiler** if VITE\_MAPTILER\_KEY is set, else CARTO / OpenStreetMap.

## 11. Mobile app (Flutter)

- **Public:** GPS nearest bins, fill + risk, OSM route preview, Google Maps navigation

- **Staff (JWT):** Dashboard, Bins, Routes, Alerts, More

Collection routes use the backend plan + **OSRM** driving geometry. Backend URL is set in `api_client.dart`.

## 12. Deploy (Railway)

Typical services:

1. `services/waste-api`
2. `services/animal-api`
3. `model-yolo`
4. optional `services/litter-severity-api`
5. backend — `MODEL_WASTE_URL`, `MODEL_ANIMAL_URL`, optional `MODEL_YOLO_URL` / `MODEL_LITTER_URL`, `DATABASE_URL`, `JWT_SECRET`, `CORS_ORIGIN`
6. frontend — build-time `VITE_API_URL`
7. Postgres plugin
8. Optional `OPENWEATHER_API_KEY` (otherwise weather is stubbed)

Always use **full URLs with https://**. Host-only strings are normalized to HTTPS.

Local ports: YOLO **6000** → backend **5000** → frontend **3000**.

## 13. Important constraints

- **LAN vs cloud:** ESP32 is only reachable from the same Wi-Fi; the laptop bridge is required for production.
- **Binary waste:** no mixed-waste CRITICAL case.
- **Forecast honesty:** does not predict new animals or cleaning crews.
- **Bin-fill model** is the weakest of the four vision models.
- **DB optional:** inference works without Postgres; history / map / alerts need it.
- **Bridge lock:** if a bin has `bridge_instance_id` set, only that laptop (or mobile/admin with `device_id`) can attach captures.
- **Holiday demo data** (`holiday_cache.json`) is for tourism/demo fill levels, not live cameras.

## 14. Repository map

| Folder                    | Role                                         |
|---------------------------|----------------------------------------------|
| <code>frontend/</code>    | Canonical React dashboard                    |
| <code>backend/</code>     | Express API gateway + risk + alerts          |
| <code>services/</code>    | waste / animal / litter Docker microservices |
| <code>model-yolo/</code>  | Bin-fill Flask service + best.pt             |
| <code>VisionWaste/</code> | ESP32 firmware helpers + laptop bridge       |
| <code>mobile_app/</code>  | Flutter Android client                       |
| <code>dataset/</code>     | Bin-fill training data                       |

| Folder                                                               | Role                        |
|----------------------------------------------------------------------|-----------------------------|
| waste_classification/, animal_detection/, litter_severity_detection/ | Training scripts + datasets |
| evaluation_results/                                                  | Paper-ready metrics         |
| web/                                                                 | Deprecated — do not deploy  |

## 15. One-line summary

---

Cameras and phones → Express gateway → four ML services + weather → risk rules → Postgres, alerts, speakers, maps, and collection routes, with a citizen landing page and a staff dashboard / mobile app.

---

Generated from the VisionWaste (R26-IT-084) codebase. This is a technical overview for the project team, not a substitute for the live README files in each service folder.